



UniAssist

Centralized AI Student Portal — MAJU

Project Technical Guide

End-to-end technical samajh — har component, har number, poorā flow (Roman Urdu mein, standard definitions ke saath)

Project Team	Registration
Syed Ibrahim	SP23-BSCS-0178
Mubashir Riaz	SP23-BSCS-0208
Adnan Kamal	SP23-BSCS-0179
Abdul Rehman	SP23-BSCS-0181

Program: BS Computer Science | June 15, 2026

Is guide mein kya hai

1. Project ka taaruf — standard definition jo hum sab istemaal karenge
2. System Architecture — teen tiers aur unka kaam
3. Knowledge Base kaise banta hai — Build-time pipeline (5 steps)
4. Embedding kya hai — text se 'meaning vector' tak
5. Live Query Flow — har sawaal pe kya hota hai (dil of the project)
6. Retrieval ko gehrai se samjho — Semantic + Keyword (Hybrid)
7. Prompt aur LLM — jawab kaise banta hai
8. LLM Stack — Groq (primary) + dual-key failover + Gemini
9. Roman Urdu support — 4 layers
10. Voice, Admin knowledge editor, Moderation & Security
11. Numbers Cheat-Sheet — yaad rakhne wale aankde
12. Glossary — standard technical terms (yehi alfaaz hum bolenge)

Is guide ko kaise istemaal karein

Sections **3, 4, 5 aur 6** project ka **technical core** hain — inhe itni baar parho ke apne alfaaz mein samjha sako. Section **12 (Glossary)** ko 'standard definition' ke taur pe use karo: jab bhi koi term aaye (embedding, chunk, RAG, top-k...) to *wohi* definition sab bolein — taake poori team consistent rahe.

1. Project ka Taaruf

Standard ek-line definition (yehi sab bolenge)

“UniAssist ek full-stack, AI-powered student portal hai Muhammad Ali Jinnah University ke liye, jiska core ek **Retrieval-Augmented Generation (RAG)** chatbot hai — yeh MAJU ki apni website ke data par grounded reh kar English aur Roman Urdu (voice samet) mein sawaalon ke jawab deta hai — aur iske saath role-based **Student / Staff / Admin** portals aur ek **issue-tracking** system hai.”

Masla kya hai

MAJU ki maloomat aur student services **bikhri hui** hain. Jawab kai web pages par phaile hain; madad ke liye office jaana ya email karna padta hai jiska jawab shayad na aaye. Koi aisi jagah nahi jo hamesha available ho, student-friendly ho, aur us tarah samajh sake jaise Pakistani students asal mein likhte hain (Roman Urdu, mixed language, chhoti SMS-style spelling).

Hamara hal

- **AI Chatbot** — MAJU ke sawaalon ke jawab, MAJU website par grounded, English / Roman Urdu / mixed mein, voice input ke saath.
- **Issue Tracker** — student kisi department (SFC, IT, HOD) ko ticket bhejta hai; staff resolve karta hai; student ko email + in-app bell se notify hota hai.
- **Role-based portals** — alag, secured experience har role ke liye: **Student, Staff, Admin**.
- **Admin control** — users, staff, departments, analytics, aur chatbot ka knowledge base bhi live edit ho sakta hai.

Sab se aham technical baat

Is project ka **asal dimagh** RAG chatbot hai. Baqi cheezein (portals, issue tracker) standard web development hain — lekin RAG pipeline (scraping, chunking, embedding, hybrid retrieval, grounding) wo hissa hai jise samajhna aur samjhana sab se zaroori hai. Is guide ka zyada hissa usi par hai.

2. System Architecture

UniAssist ek **three-tier system** hai — teen alag services jo HTTP par baat karti hain. Har tier apne kaam ka best tool istemaal karta hai aur alag se develop/scale ho sakta hai.

FRONTEND (client/)	BACKEND (server/)	AI SERVICE (python/)
React 19 + Redux Toolkit Tailwind + Vite Student / Staff / Admin UI	Node.js + Express MongoDB + Mongoose Auth, APIs, RBAC, audit, notifications, file uploads	FastAPI + Uvicorn ChromaDB + LLM client RAG chatbot, moderation, language detect, Whisper STT

Do data stores hain: **MongoDB** mein users, issues, chats, notifications aur audit/login logs hain. **ChromaDB** (ek vector database) mein chatbot ka knowledge hai — MAJU website se nikale gaye **4,763 text chunks**.

Tiers aapas mein kaise baat karti hain

- **Browser → Node:** React app Express API ko call karta hai (JWT header mein) — login, issues, admin, notifications sab ke liye.
- **Browser/Node → Python:** chat sawaal aur voice notes FastAPI service ko jaate hain (/ask, /transcribe); admin ka knowledge editor seedha Python ke /chunks ko call karta hai.
- **Python → Node (internal webhook):** jab bot ko gaali/abuse detect ho to wo Node ke /internal/flag-user ko shared secret ke saath call karta hai — student flag ho jata hai aur admin ko alert milta hai.

Python ko alag service kyun rakha?

AI/ML ka poora ecosystem — embeddings, ChromaDB, Whisper, LLM clients — **Python-first** hai. Inhe Node mein ghusane ke bajaye humne ek FastAPI microservice mein alag kar diya. Node saaf business-logic server rehta hai; Python 'AI dimagh' hai. Dono HTTP par loosely-coupled hain, to kisi ek ko badle bagair doosre ko swap/scale kiya ja sakta hai.

3. Knowledge Base kaise banta hai

Chatbot ko jawab dene ke liye pehle **knowledge** chahiye. Yeh ek **build-time pipeline** se banta hai jo sirf ek baar (ya jab data badle) chalta hai. Paanch steps hain: **Scrape** → **Clean** → **Chunk** → **Embed** → **Store**. Har step neeche detail mein hai.

```
MAJU Website (jinnah.edu)
| (1) Scrape -> sitemap se URLs, 16 parallel workers
v
Raw HTML pages
| (2) Clean -> nav/footer/script/ads hata do
v
Saaf structured text (headings + paragraphs)
| (3) Chunk -> chhote topic-blocks (~900 chars)
v
4,763 chunks
| (4) Embed -> har chunk -> 384 numbers (vector)
v
Vectors + text + metadata
| (5) Store -> ChromaDB (collection: university_chunks)
v
Searchable Knowledge Base (ready)
```

Step 1 — Scrape (web se data uthana)

File: scraper.py. MAJU ke sitemap (jinnah.edu/sitemap_index.xml) se saare page URLs nikale jaate hain. Phir **16 parallel workers** (threads) ek saath pages download karte hain taake tez ho. Har request ka timeout **10 seconds** hai.

Faaltu URLs pehle hi chhaant diye jaate hain: images/PDF jaisi files, aur WordPress ke 'listing' pages (author archives, tag/category indexes, paginated feeds) — kyunki inme asal maloomat nahi hoti, sirf doosre pages ki list hoti hai.

Step 2 — Clean (kachra hatana)

Har page se sirf **asal article content** rakha jaata hai. BeautifulSoup se HTML parse hota hai aur yeh cheezein nikaal di jaati hain: script, style, nav, header, footer, sidebars, share buttons, comments, related-posts widgets waghaira. Yeh saaf text hi aage jaata hai.

Junk detection (smart filter)

Sirf tags hatana kaafi nahi. Kuch blocks 'widget kachra' jaise hote hain — jaise naamon ki list (ALL CAPS) ya taareekhон ka dhaer ('recent posts'). Ek heuristic check (sentence density, all-caps ratio, date count) aise blocks ko junk maan kar nikaal deta hai — lekin agar usme email/phone/fee jaisi **important info** ho to use rakh leta hai.

Step 3 — Chunk (bade text ko chhote topic-blocks mein todna)

Yeh sab se aham step hai. Hum text ko **raw character count** se nahi todte — balke **structure** ke hisaab se. Ek chunk = **ek heading + uske neeche ke paragraphs** (ek self-contained topic). Iska faida: har chunk ka apna mukammal matlab hota hai.

Exact rules (config.py se):

Setting	Value	Matlab
TARGET_CHUNK_SIZE	900 chars	Soft target — paragraphs is hudd tak pack karo
MAX_CHUNK_SIZE	1600 chars	Agar ek section bada ho to sentence-boundary pe toro (paragraph kabhi beech se nahi katta)
MIN_CHUNK_LENGTH	120 chars	Is se chhota chunk agle ke saath merge ho jata hai

Ek aur key trick: **heading har chunk ke text ke aage laga di jaati hai**. Misaal: 'Admission Requirements' + uska paragraph. Isse jab embedding banti hai to usme topic ka matlab bhi aa jata hai — search behtar hota hai.

Step 4 — Embed (text ko numbers mein badalna)

Har chunk ko **embedding model** (all-MiniLM-L6-v2) se guzaar kar ek **384 numbers ka vector** banaya jaata hai jo us text ka *matlab* capture karta hai. Yeh batch mein hota hai (**100 chunks ek waqt**). (Embedding ki tafseel agle section mein.)

Step 5 — Store (ChromaDB mein mehfooz karna)

Har chunk in 4 cheezon ke saath ChromaDB ki `university_chunks` collection mein save hota hai:

- **id** — unique pehchaan (jaise `chunk_42`)
- **document** — asal text (page title + heading + paragraph)
- **metadata** — source URL aur `chunk_id`
- **embedding** — wo 384-number vector

Bas — ab knowledge base search ke liye taiyaar hai. Yeh poora pipeline `python run.py scrape` aur `python run.py build` se chalta hai.

4. Embedding kya hai (gehrai se)

Embedding poore RAG ka **buniyadi idea** hai. Ek baar yeh samajh aa jaye to baqi sab aasaan hai.

Masla: computer alfaaz ka 'matlab' nahi samajhta

Normal database sirf **exact lafz** match karta hai. Agar aap 'tuition cost' search karein to wo 'fee structure' wala page nahi dhoondh payega — kyunki lafz alag hain, magar matlab ek hai. Hume **matlab** ke hisaab se search chahiye.

Hai: har text ko numbers ke vector mein badlo

Embedding model ek aisi trained AI hai jo kisi bhi text ko **384 numbers ki list** (vector) mein badal deti hai. Khoobi yeh hai: **milte-julte matlab wale text ke vectors aapas mein qareeb hote hain**, aur alag matlab wale door. Yaani matlab ab 'geometry' ban gaya — fasla naapa ja sakta hai.

```
"BSCS ki fee kitni hai?" -> [0.12, -0.04, 0.88, ... ] (384 numbers)
"computer science fees" -> [0.11, -0.05, 0.85, ... ] <- qareeb (similar)
"hostel ka khana kaisa hai" -> [-0.71, 0.33, 0.02, ... ] <- door (alag topic)
```

Hamara embedding model: all-MiniLM-L6-v2

Cheez	Tafseel
Naam	all-MiniLM-L6-v2 (Sentence-Transformers se)
Output	384 numbers (dimensions) har text ke liye
Size	Sirf ~27 MB — chhota aur tez, CPU pe chalta hai
Kaam	Text ka semantic (matlab wala) vector banana
Kahan local	Yeh model hamari machine pe chalta hai — koi cloud call nahi

Query bhi isi tarah embed hoti hai (sab se aham baat)

Build-time pe humne **chunks** embed kiye the. Jab koi student sawaal poochta hai, to **wahi embedding model us sawaal ko bhi 384-number vector mein badalta hai** — bilkul usi tareeqe se. Ab dono ek hi 'space' mein hain, to ChromaDB sirf yeh dekhta hai: sawaal ke vector ke sab se qareeb kaunse chunk vectors hain? Wahi sab se relevant chunks hain.

Similarity kaise naapte hain — cosine distance

Do vectors kitne qareeb hain, yeh **cosine similarity** se naapa jaata hai (angle between vectors). ChromaDB 'distance' deta hai; hum **score = 1 – distance** use karte hain — score jitna zyada (1 ke qareeb), text utna hi relevant. Yeh number aage ranking mein kaam aata hai.

5. Live Query Flow — har sawaal pe kya hota hai

Yeh project ka **dil** hai. Jab student ek sawaal bhejta hai, wo neeche diye **8 steps** se guzarta hai. Examiner is flow ko zaroor poochega — ise zabani yaad rakho.

```
Student ka sawaal
|
(1) Language detect -> English / Roman Urdu / mixed?
(2) Moderation -> gaali/abuse hai? -> block + flag
(3) Retrieval query -> follow-up hai to context jodo
(4) Hybrid retrieval -> Semantic + Keyword -> top 8 chunks
(5) Prompt banao -> chunks + rules + sawaal
(6) LLM call -> Groq (primary) jawab likhta hai
(7) Cleanup -> filler hatao, language guards
(8) Jawab wapas -> chat mein dikhao
```

Step 1 — Language Detection

File: `language_detect.py`. Sawaal ko classify kiya jaata hai: **English, Roman Urdu**, ya **mixed**. Yeh `langdetect` library aur ek custom **Roman-Urdu function-word lexicon** se hota hai (SMS-style text jaise 'fees kitni hai' ke liye). Detected language aage LLM ko batati hai ke kis zabaan mein jawab dena hai.

Step 2 — Moderation (abuse filter)

File: `moderation.py`. English + Roman-Urdu gaaliyon ke liye scan hota hai. Agar message abusive ho: jawab nahi banta, student ko politely mana kiya jaata hai, uska record **flag** ho jata hai, aur admin ko internal webhook se alert jata hai.

Step 3 — Retrieval Query banana (follow-ups ke liye)

File: `rag.py` → `_build_retrieval_query()`. Yeh ek **aham aur naya fix** hai. Masla yeh tha: agar har follow-up ke saath pichla sawaal jod diya jaye, to mukammal sawaal (jaise 'Admission requirements?') ka search kharab ho jata tha — pichle topic (programs) ke chunks upar aa jaate the aur ghalat jawab milta tha.

- Ab pichla sawaal **sirf tab** joda jata hai jab sawaal **adhoora (elliptical)** ho — jaise 'or fee?', 'aur iska deadline?', 'its fee?' (connector se shuru ya sirf pronoun).
- Mukammal sawaal ('Admission requirements?', 'Application deadlines?') **standalone** search hota hai — koi dilution nahi.
- Safety net: zaroorat pade to dono (standalone + context-wala) retrievals merge ho jate hain.

Step 4 — Hybrid Retrieval

Top **8** sab se relevant chunks dhoonde jaate hain — do tareeqon ko mila kar (Semantic + Keyword). Yeh itna aham hai ke iska poora ek section hai (Section 6).

Step 5 — Prompt banana

Mile hue chunks ko ek **prompt** mein 'context' ke taur pe daala jaata hai, kuch sakht rules ke saath: sirf context se jawab do, kuch invent mat karo, detected zabaan mein jawab do, kabhi yeh mat batao ke kaunsa AI model ho. (Tafseel Section 7.)

Step 6 — LLM call (jawab generate karna)

Prompt **Groq** (primary cloud LLM) ko bheja jaata hai jo jawab likhta hai. Temperature **0.3** (focused, kam random), max **1000 tokens**. Agar Groq ki ek key ka limit khatam ho to doosri key, phir Gemini — automatic failover (Section 8).

Step 7 — Cleanup aur language guards

- Filler hatao: 'Sure!', 'Based on the context...' jaise preambles aur [1]/[2] citation markers nikaal diye jaate hain.
- Roman Urdu guards: agar model Hindi/Devanagari ya Arabic script likh de to ek baar dobara koshish, phir wo characters strip; aur leaked Hindi alfaaz Urdu se badle jate hain (Section 9).

Step 8 — Jawab wapas

Final saaf jawab chat mein chala jaata hai, aur conversation history mehfooz rehti hai taake agla follow-up samajh aaye.

Poore flow ka ek-jumla version

"Sawaal → zabaan pehchaano → abuse check → MAJU knowledge mein search (semantic + keyword) → behtareen 8 chunks ko prompt mein daalo → Groq jawab likhta hai → saaf karo aur zabaan theek karo → jawab."

6. Retrieval ko gehrai se samjho (Hybrid)

File: `database.py` → `search()`. Hum do alag tareeqon se chunks dhoondte hain aur unhe **merge** kar dete hain. Isko **Hybrid Retrieval** kehte hain.

Tareeqa A — Semantic Search (matlab wala)

Sawaal ko embed karke (384-vector) ChromaDB se cosine ke hisaab se sab se qareeb chunks liye jaate hain. Iska score = $1 - \text{distance}$. Yeh paraphrase pakadta hai — 'cost of CS' bhi 'BSCS fee structure' dhoondh leta hai.

Tareeqa B — Keyword Search (exact lafz wala, TF-IDF)

Semantic kabhi-kabhi **exact tokens** ko kam ahmiyat deta hai — jaise course code 'CS-3413', phone number, ya koi makhsoos fee figure. Iske liye ek **TF-IDF based keyword pass** chalta hai. Iske andar yeh sab hota hai:

- **Tokenization + stopwords:** sawaal se faaltu alfaaz (the, is, kya, hai...) hata kar sirf ma'ane-khez tokens rakhe jaate hain.
- **Stemming:** 'programs/programme' → 'program', 'admissions' → 'admission' — taake variations match karein.
- **Abbreviation expansion:** 'bscs' → bs, cs, computer, science; 'mba' → business, administration — taake short form bhi mil jaye.
- **IDF weighting:** jo lafz puri corpus mein kam aata hai usse zyada weight milta hai (zyada specific = zyada ahmiyat). Formula: $IDF = \log((N+1)/(df+1))$.
- **URL boost:** agar lafz chunk ke source URL mein bhi ho to uska score 1.5x ho jata hai.

Dono ko Merge karna

Ab dono lists ko jod kar **combined score** nikala jaata hai. Same chunk dono mein ho to use ek hi baar rakhte hain (dedup — pehle 120 characters se pehchaan). Final formula:

```
combined_score = semantic_score + 1.0 x keyword_score

-> dedup (same chunk ek baar)
-> combined_score ke hisaab se sort
-> top 8 chunks le lo (TOP_K_RESULTS = 8)
```

Natija: hume **matlab** bhi milta hai aur **exactness** bhi — dono ka best. Yeh 8 chunks prompt ka 'context' bante hain.

Agar poocha jaye: ‘Hybrid kyun, sirf semantic kyun nahi?’

“Semantic search matlab ke liye behtareen hai lekin exact tokens (course code, phone, fee figure) ko kabhi miss kar deta hai. Keyword/TF-IDF pass wo pakadta hai. Dono ko merge karke hum meaning aur exactness dono haasil karte hain — yeh retrieval pipeline humne khud banaya hai.”

7. Prompt aur LLM — jawab kaise banta hai

Retrieval ne 8 chunks de diye. Ab inhe ek **prompt** mein daal kar LLM se jawab likhwaya jaata hai. Prompt ka design hi tay karta hai ke jawab achha aayega ya bura.

Prompt mein kya hota hai

- **System message:** 'Tum MAJU Assistant ho... facts invent mat karo, kabhi AI model ka naam mat batao' + detected zabaan ki hidayat.
- **Context:** wo 8 retrieved chunks (Source 1, Source 2, ...).
- **Chat history:** pichle turns (max 6) taake follow-up samajh aaye.
- **Student ka sawaal** + jawab dene ke rules.

Jawab ki 6 'states' (smart prompt design)

Prompt model ko pehle yeh faislaa karne ko kehta hai ke sawaal kis *qism* ka hai, phir usi branch ko follow kare. Yeh 'ek qism chuno' tareeqa '9 rules ek saath' se behtar chalta hai:

State	Sawaal ki qism	Jawab ka tareeqa
A	Greeting / salam / shukriya	Garmjoshi se 1-2 jumle, context ignore
B	Identity ('tum kaun ho?')	'Main MAJU Assistant hoon' — koi AI model ka naam nahi
C	Is chat ke baare mein ('mne kya pocha')	Chat history se jawab, context ignore
D	Off-topic (mausam, doosri university)	Politely mana karo, MAJU pe wapas lao
E	MAJU sawaal (fee, admission, course...)	Context + history se seedha jawab; invent mat karo
F	Draft/likhne ki request (email likho)	Mukammal cheez likh do, placeholders ke saath

Anti-hallucination (sab se aham rule)

Prompt sakhti se kehta hai: agar context aur history dono mein jawab na ho to **invent mat karo** — balke saaf keh do 'mere paas iski maloomat nahi'. Isi liye bot ghalat fee/deadline banane ke bajaye sach bolta hai. Yeh RAG ki sab se badi taqat hai.

LLM settings

Setting	Value	Matlab
Temperature	0.3	Kam = focused, factual jawab (zyada = random/creative)
Max tokens	1000	Jawab ki maximum lambai
Top-k (chunks)	8	Kitne knowledge chunks model ko diye jaate hain
History cap	6 messages	Pichle 6 turns hi bheje jaate hain (prompt chhota rahe)

8. LLM Stack — Groq + Failover

Jawab likhne wala ‘dimagh’ (LLM) ab **Groq cloud** hai — apni answer quality aur instruction-following ki wajah se. System resilient hai: agar ek option fail ho to khud doosre pe chala jata hai.

Provider order (failover chain)

```
USE_LOCAL_LLM = false -> cloud mode (current default)
CLOUD_LLM_ORDER = groq, gemini

1. Groq - key #1 (primary account)
| agar 429 (rate/limit khatam) ya error
v
2. Groq - key #2 (doosra account) <- dual-key failover
| agar wo bhi fail
v
3. Gemini (aakhri fallback)
```

Dual-key failover (naya feature)

Groq free-tier ki rate limit hoti hai. Isliye hum **do alag accounts ki API keys** rakhte hain. Jab pehli key ka limit (HTTP 429) khatam ho jaye, code **khud-ba-khud doosri key** pe roll over kar deta hai — student ko pata bhi nahi chalta. Keys .env mein hain: GROQ_API_KEY, GROQ_API_KEY_2 (aur chaho to _3).

Models

Provider	Model	Role
Groq	llama-3.3-70b-versatile	Primary — 70B model, tez aur achhe jawab
Gemini	gemini-2.5-flash	Last-resort fallback
Ollama (local)	llama3.2	Optional — agar USE_LOCAL_LLM=true ho (offline/private)

Local option abhi bhi maujood hai

Ek hi config flag (USE_LOCAL_LLM=true) se project **fully local Ollama** pe chala jaata hai — koi cloud call nahi, data machine se bahar nahi jata. Yaani hum cloud ki quality aur local ki privacy, dono ke beech ek line mein switch kar sakte hain. Yeh design ki lachak (flexibility) dikhata hai.

9. Roman Urdu Support — 4 Layers

Pakistani students Roman Urdu mein likhte hain (English letters mein Urdu). Iska sahi jawab dena aasaan nahi — model kabhi Hindi/Devanagari script likh deta hai ya Hindi alfaaz mila deta hai. Isliye hum **4 layers** ka defence rakhte hain.

Layer	Kya karta hai
1. Detection	Sawaal ki zabaan pehchaano (langdetect + Roman-Urdu lexicon)
2. Prompt instruction	Model ko saaf kaho: Roman Urdu mein, sirf English letters, Urdu vocabulary (Hindi nahi). Achhi/buri misaalein bhi di jaati hain.
3. Script guard	Agar jawab mein Devanagari/Arabic script aa jaye to ek baar dobara koshish; phir bhi rahe to wo characters strip kar do.
4. Vocabulary guard	Leaked Hindi alfaaz ko Urdu se badlo: sampark→raabta, vah→woh, dhanyavad→shukriya, prashn→sawaal.

Kyun zaroori hai

Roman Urdu aur Roman Hindi gareeb lagte hain lekin alag hain (Urdu = Persian/Arabic-origin alfaaz, Hindi = Sanskrit-origin). Ek 3B/chhota model aksar inhe mix kar deta hai. Yeh layered approach jawab ko natural aur student-friendly rakhta hai — bina kisi mehngi training ke.

10. Voice, Admin Editor, Moderation & Security

Voice Input (Whisper STT)

Student awaaz mein sawaal kar sakta hai. Browser audio record karta hai → Node use Python ke `/transcribe` ko bhejta hai → **faster-whisper** (model 'base', CPU, int8) use **locally** text mein badalta hai → wahi text phir usi RAG pipeline se guzarta hai. Koi paid cloud STT (jaise AssemblyAI) nahi — sab local.

Live Knowledge-Base Editor (standout feature)

Admin panel se admin chatbot ka **dimagh live edit** kar sakta hai — bina kisi code change ya retraining ke. File: `api.py` ke admin endpoints.

- **Chunks CRUD:** chunks list/search/add/edit/delete (`/chunks`). Har edit pe text dobara embed hota hai aur cache clear hota hai.
- **Document upload:** PDF/DOCX/TXT upload karo (`/documents`) — wo automatically chunk + embed ho kar knowledge base mein add ho jaata hai.
- Yaani naya program/policy add karna ho to bas ek document upload — bot foran us baare mein jaan jata hai.

Moderation & Security

- **Moderation:** har sawaal pehle English + Roman-Urdu gaali ke liye check hota hai; abusive ho to block + student flag + admin alert.
- **Admin auth:** Python admin endpoints JWT verify karte hain aur Node se confirm karte hain ke user waqai admin hai (`verify-admin`) — phir hi knowledge edit allow.
- **Service-to-service:** Node aur Python ke beech internal endpoints ek shared `INTERNAL_SECRET` header se mehfooz hain — bahar wala koi call nahi kar sakta.
- **Secrets:** API keys aur secrets `.env` mein hain jo git se ignore hai (repo mein push nahi hote).

11. Numbers Cheat-Sheet

Yeh aankde zabani yaad karo — examiner aksar inhe poochta hai.

Cheez	Value
LLM (primary)	Groq — llama-3.3-70b-versatile (cloud)
LLM failover	Groq key#1 → key#2 → Gemini (gemini-2.5-flash)
Local option	Ollama llama3.2 (USE_LOCAL_LLM=true)
Embedding model	all-MiniLM-L6-v2 — 384 dimensions, ~27 MB
Vector DB	ChromaDB — collection 'university_chunks'
Knowledge size	4,763 chunks (MAJU website se)
Retrieval	Hybrid (semantic + keyword TF-IDF), top-k = 8
Score formula	semantic (1 – distance) + 1.0 x keyword
Chunk size	~900 target, 1600 max, 120 min (characters)
LLM settings	temperature 0.3, max 1000 tokens, history 6 turns
Voice (STT)	Whisper 'base' via faster-whisper (local, CPU, int8)
Languages	English, Roman Urdu, mixed (+ 4-layer RU guard)
Stack	React 19 / Node+Express / FastAPI; MongoDB + ChromaDB

Standard one-liners (yehi jumle bolo)

- “RAG ka matlab: jawab **MAJU ke apne data** se aata hai, model ke khayal se nahi.”
- “Embedding text ko **384 numbers** mein badalta hai taake matlab ke hisaab se search ho.”
- “Retrieval **hybrid** hai — semantic (matlab) + keyword (exact lafz).”
- “Anti-hallucination: context mein na ho to bot saaf keh deta hai ‘maloomat nahi’.”
- “Groq primary hai, do keys ka failover + Gemini fallback — system kabhi ruk-ta nahi.”

12. Glossary — Standard Definitions

Yeh hamari **standard definitions** hain. Jab bhi yeh term aaye, poori team *yehi* baat bole — taake explanation consistent rahe.

Term	Standard definition (yehi bolenge)
RAG	Retrieval-Augmented Generation — pehle relevant data dhoondo (retrieve), phir LLM se usi data par jawab likhwao (generate). 'Open-book exam' jaisa.
Embedding	Kisi text ko numbers ke vector (yahan 384 numbers) mein badalna jo us text ka matlab capture karta hai.
Vector	Numbers ki list jo text ka matlab represent karti hai. Milte-julte matlab ke vectors qareeb hote hain.
Chunk	Knowledge ka ek chhota self-contained tukda (heading + paragraph, ~900 chars). Search inhi par hota hai.
Vector Database	Aisa database jo vectors store karta hai aur matlab (similarity) ke hisaab se search karta hai. Hum ChromaDB use karte hain.
Semantic Search	Matlab ke hisaab se search (exact lafz zaroori nahi) — embeddings + cosine similarity se.
Keyword / TF-IDF	Exact lafz match; kam aane wale lafz ko zyada weight (IDF). Exact tokens (codes, fees) pakadta hai.
Hybrid Retrieval	Semantic + keyword dono ko merge kar ke best chunks lena.
Top-k	Search se kitne sab se relevant chunks lene hain (hamare yahan $k = 8$).
Temperature	LLM ki randomness (0-1). Kam (0.3) = focused, factual jawab.
Token	LLM ke liye text ka unit (taqreeban ek lafz/lafz ka hissa). Max tokens jawab ki lambai control karta hai.
Hallucination	Jab LLM aatmvishwas se ghalat baat bana de. RAG + sakht prompt se hum ise rokte hain.
Failover	Jab ek service/key fail ho to khud-ba-khud doosri par switch ho jana (Groq key#2, phir Gemini).
Grounding	Jawab ko asal source (MAJU data) se baandhna taake wo verifiable aur sahi rahe.

Asli UniAssist codebase (python/) se verify kiya gaya. Yeh team ki standard technical reference hai.